

epoch_info chain & snapshots — test scenarios

Feature branch `feat/move-epoch-info-chain-into-checkpointstore` (PR #11996). Risk is concentrated in the startup state matrix, which unit/sim tests only partially cover.

0. Automated first (cheap — run before any manual testing)

- `cargo simtest -p iota-e2e-tests epoch_info` — genesis-rebuild + parity, snapshot-verify, tampering, safe-mode, residual-gap-above-prefix, live-population.
- `cargo simtest -p iota-snapshot` — EPOCH_INFO round-trip, field-order lock, chain-verify rejects.
- These cover the *logic*. Everything below is about real DB/network states they can't simulate.

1. Upgrade-in-place matrix (the core migration risk)

Existing DB built by the **old** binary (no `epoch_info` table) → start the **new** binary. Vary history depth and pruning.

#	Starting DB	Network	Expected
1a	Full history since genesis, no pruning, caught up to tip	mainnet/testnet	Rebuilds whole chain locally from genesis (after snapshot fetch — see note); starts.
1b	Pruned history (typical fullnode), caught up to tip	mainnet/testnet	Snapshot fills bulk, local fills residual tail; starts.
1c	Pruned history, ~2 days behind tip (your example)	mainnet/testnet	Snapshot (lags a bit) + local residual; then state-sync catches up. Starts. Watch: does the published snapshot cover the gap, or is there a hole pruned-local can't fill?
1d	Pruned history, far behind (weeks), snapshot also lags	mainnet/testnet	Residual gap → fatal panic. Verify the error message is clear and actionable.
1e	Validator (not fullnode), pruned	mainnet/testnet	As 1b/1d — now stricter: a validator with a gap that used to start (warn) now panics. Confirm intended.
1f	Snapshot bucket unreachable at startup (firewall/DNS)	mainnet/testnet	Recognized chain → fatal. Confirm it fails comprehensibly, not a hang.

Note on 1a: with the snapshot-first reorder, a full-history recognized node now downloads the snapshot anyway rather than rebuilding purely locally. Confirm that's acceptable startup latency/bandwidth.

2. Formal-snapshot restore (the V2 path)

- **2a.** Restore from a new V2 formal snapshot (has EPOCH_INFO) → start. Chain complete from the restore itself, no startup backfill.
- **2b.** Restore from V2, then start with `--skip-grpc-indexes` → chain still complete (decoupled from gRPC).
- **2c.** Restore from an old V1 snapshot (pre-feature, no EPOCH_INFO) → start new binary → must backfill from the hardcoded bucket. (Confirm a V1 restore path still runs.)
- **2d.** Restore a snapshot whose epoch lags the bucket's latest → backfill brings it forward.

- **2e.** Restore from a RocksDB snapshot (not formal) → confirm epoch_info ends up complete (which path seeds it?).

3. Snapshot production (the write side)

- **3a.** Snapshot-writing node with `enable_grpc_api` OFF produces a formal snapshot → confirm EPOCH_INFO is present and complete (the headline decoupling).
- **3b.** The snapshot from 3a, restored by a fresh node (→ loop to 2a) → chain verifies and is complete. **This round-trip is the single most important end-to-end check.**
- **3c.** Snapshot-writing node with an incomplete chain → refuses to start (fatal) before it can publish a gapped EPOCH_INFO.
- **3d.** Produced EPOCH_INFO chain-verifies against the genesis committee (run `verify_epoch_info_chain` over a real produced file).

4. Consumers reading the chain

- **4a.** gRPC `GetEpoch` / epoch queries return correct data, reading from `CheckpointStore` (not the old gRPC index). Spot-check historical + current.
- **4b.** A node in gap-warn state (unknown network) → `GetEpoch` for a missing epoch behaves sanely (no crash).
- **4c.** Reconfiguration across a boundary works (reads `epoch_last_checkpoint_map` for the current epoch) — run through ≥ 1 boundary on the new binary.

5. Network recognition & fatal/warn semantics

- **5a/5b.** mainnet / testnet — recognized, fatal on gap. ✓ via §1.
- **5c.** devnet, chain id matches the hardcoded one → recognized, uses devnet bucket, fatal on residual gap. **Highest-risk item: confirm the bucket name `iota-devnet-formal` is right, else a matched devnet node bricks.**
- **5d.** devnet after a reset (chain id no longer matches) → falls to unknown → best-effort, starts with a warning (does not brick). Verify directly against a fresh devnet — this is the safety property.
- **5e.** localnet / custom (unknown) → no snapshot source; local-only; residual gap → warning, starts. Confirm the warn message prints.

6. Pruning / lag edges (boundary-only assembly payoff)

- **6a.** Node that pruned object versions but kept checkpoint/effects/boundary state → local backfill still succeeds (the Option C robustness gain).
- **6b.** Node where the boundary tx's own data is pruned → that epoch can't rebuild → stops there (gap above). Graceful stop on the local side, not a panic.
- **6c.** Genesis checkpoint pruned → `try_seed_genesis_epoch` returns false → relies on snapshot.

7. Crash / restart / idempotency

- **7a.** Kill mid-backfill, restart → resumes cleanly, no double-application (idempotent upsert + watermark guard).
- **7b.** Kill mid-epoch-boundary (live indexing), restart → boundary re-applied harmlessly.
- **7c.** Steady-state restart (chain already complete) → fast path: no snapshot download, no rebuild. Confirm no network I/O on a normal restart.

8. Safe-mode & unusual boundaries

- **8a.** An epoch that closed in safe mode (`advance_epoch_safe_mode` , no events) → row finalizes with empty events, chain stays complete. (Sim test exists; confirm on a network that actually had a safe-mode epoch if one exists.)

9. Mixed-binary network (during rollout)

- **9a.** Some validators old binary, some new, across a boundary → consensus unaffected (`epoch_info` is local-only); each new node builds its own chain. Confirm no divergence.
-

Prioritization

Must-run before merge: §0 (automated), 3b (produce→restore round-trip), 1b + 1d (pruned upgrade happy + fatal), 5c/5d (devnet bucket name + reset safety), 7c (no-op restart). These exercise every distinct code path and the two scariest risks.

Strongly recommended: 1c (the 2-day case), 3a (gRPC-off production), 2a (V2 restore), 6a (pruned-object robustness).

Two things to validate out-of-band (flag in the PR)

- **Devnet bucket name** (`iota-devnet-formal`) — unverified and now on a fatal path (5c). One ops confirmation closes it.
- **Snapshot-first network dependency at startup** for recognized full-history nodes (1a) — a behavior change worth a release note.